



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0309 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

A. Likithasri (192511003)

k.kaviya (192521262)

Mary Sharlin (192511330)

Under the Supervision of

Dr.Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

A. Assignment Information

Particular	Details
Department	Department of Computer Science and Engineering
Programme	B.Tech – Artificial Intelligence and Machine Learning
Course Code & Course Name	CSA0309 – Data Structures
Academic Year / Batch	2026–2027
Faculty Name	Dr Uma Priyadarsini P.S
Assignment Title	Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the high-speed security log processing system. CO3: Make use of self-balancing trees (AVL/RedBlack Trees) for ordered, dynamically balanced storage of risk-ranked events. CO4: Make use of hashing, collision-resolution techniques, and priority queues for duplicate detection and high-risk event ranking in C. CO5: Develop a robust real-time cyber threat detection engine and evaluate its search performance under varying load factors and data distributions.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate / Create
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure; SDG 16 – Peace, Justice and Strong Institutions
Industry / Societal Relevance	Efficient processing of cybersecurity logs for rapid duplicate-IP detection, risk ranking and threat prioritization.

B. Assignment Problem / Challenge

The assignment is based on a realistic cybersecurity computing challenge involving the processing of a large number of security events in real time. The task requires students to apply data structure concepts to develop an efficient solution rather than simply reproducing theoretical concepts. The solution must support fast IP address lookup, duplicate detection, risk-based event organization, and prioritization of critical security events.

The challenge requires the selection and application of appropriate data structures such as hash tables with collision resolution, self-balancing trees, and priority queues. Students must consider practical requirements such as processing a large number of events, efficient memory usage, scalability, execution time, and the effect of different data distributions. The implemented system must also be tested and its results interpreted to justify the design decisions.

The assignment further requires the use of suitable programming and development tools for implementation, testing, benchmarking, and visualization. Performance results are analyzed using factors such as load factor, collisions, insertion time, search time, and longest chain, allowing the effectiveness of the proposed solution to be evaluated under different workloads.

C. Problem Statement

Problem / Case / Design Challenge:

Modern cybersecurity systems continuously generate large volumes of security events from different sources. Manually searching and analyzing these events can become inefficient when the number of records increases. Therefore, an efficient Real-Time Cyber Threat Detection Engine is required to process security events, identify repeated source IP addresses, organize risk information, and prioritize high-risk activities.

The design challenge is to develop a data-structure-based system capable of handling 1,000,000+ security events efficiently. The system should use hashing for fast IP lookup and duplicate detection, a self-balancing tree for maintaining risk-related information, and a priority queue for ranking events according to their risk levels.

The system must also evaluate the behaviour of the hashing technique under different load factors and data distributions, including uniform and skewed IP distributions. The performance should be analyzed using measurable parameters such as collisions, insertion time, search time, and longest chain length. The final solution should demonstrate efficient processing, scalability, and appropriate justification of the selected data structures and techniques.

D. Requirements and Constraints

Functional Requirements

- The system should accept and process cybersecurity event records.
- Source IP addresses should be stored and checked for repeated activity.
- Security events should be organized based on their assigned risk level.
- Critical events should be easily identified through priority-based processing.
- The system should provide efficient searching and retrieval of stored events.
- Performance statistics should be generated for analysis.

Performance Requirements

- The system should maintain efficient lookup and insertion as the number of events increases.
- Hashing should provide near-constant average search performance.
- AVL Tree operations should maintain logarithmic performance through balancing.
- The priority mechanism should efficiently retrieve the most critical event.
- The system should be evaluated using different event volumes and data distributions.

Technical Constraints

- The implementation should use C programming and appropriate data structures.
- Separate chaining should be used to manage hash collisions.
- An AVL Tree should be used for balanced risk-based data organization.
- A Max Heap should be used for maintaining event priorities.
- Execution time should be measured using the clock() function.
- The implementation should support benchmarking of hashing behaviour under different load conditions.

Reliability, Security and Ethical Constraints

- All incoming events should be processed without losing records during collisions.
- Duplicate activity should be identified and updated consistently.
- Risk priorities should remain accurate throughout event processing.
- The system should produce dependable results for different input conditions.
- Testing should use controlled security-event data and should be limited to authorized cybersecurity analysis.

E. Student Work

1. Problem Understanding and Formulation

The project addresses the problem of efficiently processing a large number of cybersecurity events and identifying potentially risky activities. Each event contains information such as the source IP address, event type, timestamp, risk score, and severity. As the number of events increases, searching through records sequentially becomes inefficient.

To overcome this, the implementation uses multiple data structures, with each one handling a specific requirement. The hash table provides efficient IP-based access and supports duplicate detection. Separate chaining is used to manage collisions when multiple IP addresses produce the same hash index. The AVL tree maintains events in a balanced structure based on their risk information, while the max heap keeps the highest-risk event readily accessible.

Expected Outcomes

- Efficient lookup and tracking of source IP addresses.
- Correct identification and counting of repeated IP activity.
- Balanced organization of events according to risk.
- Fast identification of the most critical security event.
- Performance evaluation under different event volumes and distributions.

Assumptions

- Risk scores are represented between 0 and 100.
- Higher risk scores indicate more serious events.
- Source IP addresses are handled as string values.
- An IP may occur multiple times in the event dataset.
- Hash collisions are handled using separate chaining.
- Testing is performed using controlled or synthetic security events.

Available Data :

Field	Example
Source IP	192.168.1.10
Timestamp	10:00:01
Event Type	Login Failed
Risk Score	90
Severity	Critical
Status	Open

2. Application of Course Knowledge

The implementation applies different data structure concepts according to the operations required by the threat detection engine.

Data Structure / Technique	Purpose in the Project	Complexity
Hash Table	IP lookup and duplicate detection	$O(1)$ average
Separate Chaining	Handles hash collisions	Depends on chain length
AVL Tree	Maintains risk-based event ordering	$O(\log n)$
Max Heap	Maintains highest-risk events	$O(\log n)$ insertion
Linked List	Stores collided entries in buckets	$O(n)$ worst case

The combination of these structures allows the system to perform different operations efficiently rather than depending on one structure for all tasks. The hash table focuses on fast identification of IP activity, the AVL tree provides balanced ordering, and the max heap provides priority-based access to critical events.

3. Solution / Design / Methodology

The system follows an event-processing approach in which each security event is passed through the required data structures. When an event is received, the source IP is converted into a hash index and the corresponding bucket is searched. If the IP already exists, its activity information is updated; otherwise, a new record is stored. This process also allows collisions to be handled without losing existing records.

The implementation additionally performs benchmarking to evaluate the behaviour of the hashing mechanism. Different numbers of events and different IP distributions can be tested, and parameters such as load factor, collision count, insertion time, search time, and longest chain can be analyzed.

Algorithm

1. Initialize the hash table, AVL tree, and max heap.
2. Read or generate a security event and extract its IP address, event type, timestamp, and risk score.
3. Calculate the hash index and search the corresponding chain for the source IP.
4. Update the existing IP record if duplicate activity is found; otherwise, create a new entry.
5. Insert the event into the AVL tree according to its risk-related key and maintain tree balance.
6. Insert the event into the max heap based on its risk score.
7. Repeat the event-processing steps until all available security events are handled.
8. Retrieve duplicate IP information and identify the highest-priority security events.
9. Perform benchmark tests using different event volumes and IP distributions.
10. Record and analyze collisions, load factor, insertion time, search time, and longest chain.

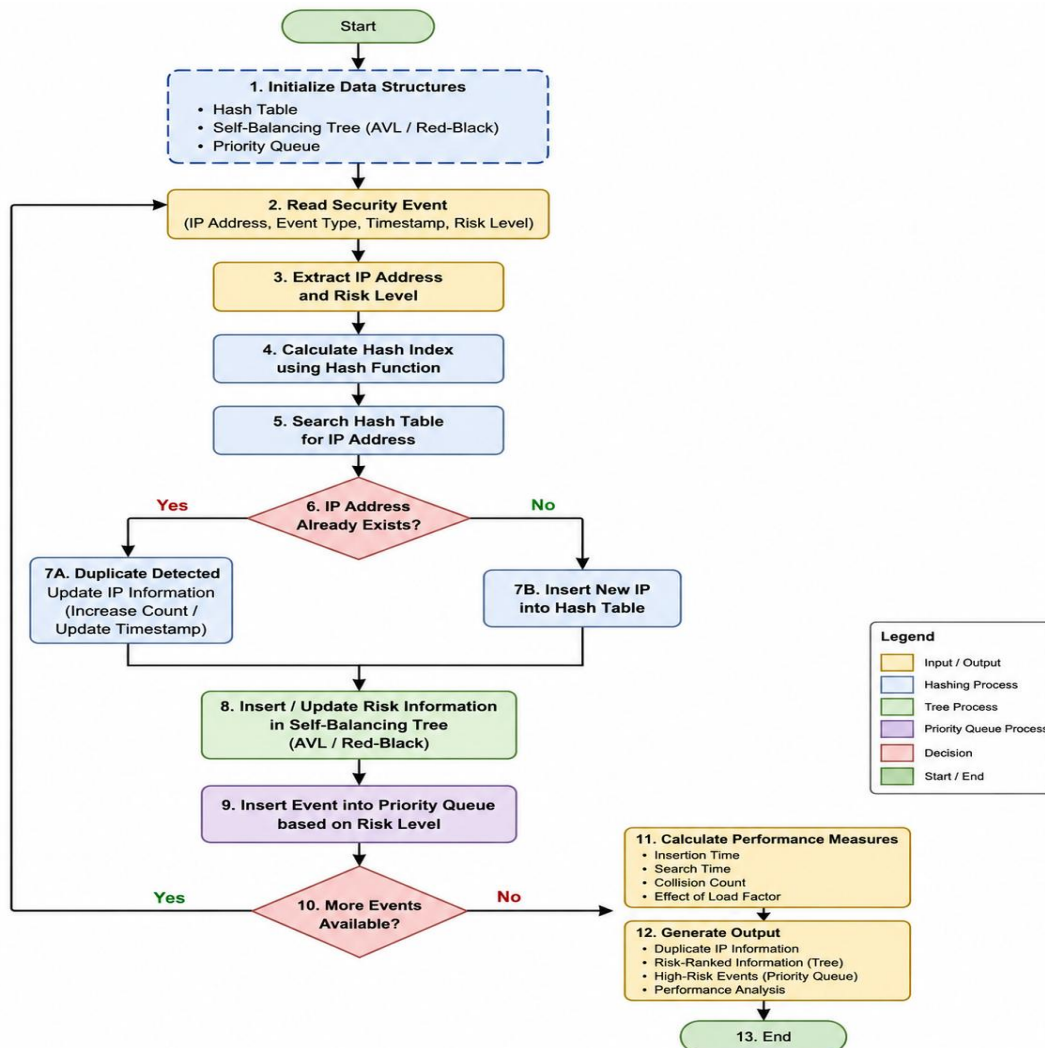


Fig : 1 Flow Chart of Cyber Security Detection Engine

Implementation Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define TABLE_SIZE 17
#define MAX_EVENTS 10000
typedef struct {
    char ip[32];
    int risk;
    char eventType[40];
    char timestamp[20];
    int id;
} Event;
typedef struct HashNode {
    char ip[32];
    int count;
    struct HashNode *next;
} HashNode;
HashNode *hashTable[TABLE_SIZE];
int hashCollisions=0;
unsigned int hashFunction(const char *ip) {
    unsigned int hash=0;
    while(*ip) {
        hash=(hash*31)+(unsigned char)(*ip);
        ip++;
    }
    return hash%TABLE_SIZE;}
void initHashTable() {
    int i;
    for(i=0;i<TABLE_SIZE;i++)
```

```

hashTable[i]=NULL;
}
void insertIP(const char *ip) {
unsigned int index=hashFunction(ip);
HashNode *current=hashTable[index];
while(current!=NULL) {
if(strcmp(current->ip,ip)==0) {
current->count++;
return;
}
current=current->next;
}
if(hashTable[index]!=NULL)
hashCollisions++;
HashNode *newNode=(HashNode*)malloc(sizeof(HashNode));
strcpy(newNode->ip,ip);
newNode->count=1;
newNode->next=hashTable[index];
hashTable[index]=newNode;
}
int findIP(const char *ip) {
unsigned int index=hashFunction(ip);
HashNode *current=hashTable[index];
while(current!=NULL) {
if(strcmp(current->ip,ip)==0)
return current->count;
current=current->next;
}
return 0;}
void printDuplicateReport() {
int i,found=0;

```



```

HashNode *current;
printf("\n--- DUPLICATE IP REPORT ---\n");
for(i=0;i<TABLE_SIZE;i++) {
current=hashTable[i];
while(current!=NULL) {
if(current->count>1) {
printf("%s : %d occurrences\n",
current->ip,current->count);
found=1;
}
current=current->next;
}
}
if(!found)
printf("No duplicate IPs detected.\n");
}

typedef struct AVLNode {
Event event;
struct AVLNode *left;
struct AVLNode *right;
int height;
} AVLNode;

int height(AVLNode *node) {
return node?node->height:0;
}

int maxInt(int a,int b) {
return a>b?a:b;
}

AVLNode *createAVLNode(Event event) {
AVLNode *node=(AVLNode*)malloc(sizeof(AVLNode));
node->event=event;

```

```

node->left=NULL;
node->right=NULL;
node->height=1;
return node;}

AVLNode *rightRotate(AVLNode *y) {
AVLNode *x=y->left;
AVLNode *temp=x->right;
x->right=y;
y->left=temp;
y->height=1+maxInt(height(y->left),height(y->right));
x->height=1+maxInt(height(x->left),height(x->right));
return x;}

AVLNode *leftRotate(AVLNode *x) {
AVLNode *y=x->right;
AVLNode *temp=y->left;
y->left=x;
x->right=temp;
x->height=1+maxInt(height(x->left),height(x->right));
y->height=1+maxInt(height(y->left),height(y->right));
return y;}

AVLNode *insertAVL(AVLNode *node,Event event) {
if(node==NULL)
return createAVLNode(event);
if(event.risk>node->event.risk)
node->left=insertAVL(node->left,event);
else
node->right=insertAVL(node->right,event);
node->height=1+maxInt(height(node->left),height(node->right));
int balance=height(node->left)-height(node->right);
if(balance>1)
return rightRotate(node);

```

```

if(balance<-1)
return leftRotate(node);
return node;}

void printAVL(AVLNode *root) {
if(root==NULL)
return;
printAVL(root->left);
printf("%-16s | Risk:%3d | %s\n",
root->event.ip,root->event.risk,
root->event.eventType);
printAVL(root->right);}

typedef struct {
Event events[MAX_EVENTS];
int size;
} MaxHeap;

void swapEvent(Event *a,Event *b) {
Event temp=*a;
*a=*b;
*b=temp;}

void heapifyUp(MaxHeap *heap,int index) {
while(index>0) {
int parent=(index-1)/2;
if(heap->events[parent].risk>=heap->events[index].risk)
break;
swapEvent(&heap->events[parent],&heap->events[index]);
index=parent;}}

void insertHeap(MaxHeap *heap,Event event) {
if(heap->size>=MAX_EVENTS)
return;
heap->events[heap->size]=event;
heapifyUp(heap,heap->size);

```

```

heap->size++;}

Event *highestRisk(MaxHeap *heap) {
if(heap->size==0)
return NULL;
return &heap->events[0];}

void performanceTest() {
int sizes[]={100,500,1000,2000,5000};
int count=5;
int s,i,n;
clock_t start,end;
printf("\n--- PERFORMANCE TEST ---\n");
for(s=0;s<count;s++) {
n=sizes[s];
start=clock();
for(i=0;i<n;i++) {
char ip[32];
snprintf(ip,sizeof(ip),"10.%d.%d.%d",
(i/65025)%255,(i/255)%255,(i%254)+1);
hashFunction(ip);}
end=clock();
printf("%d events : %.6f seconds\n",
n,(double)(end-start)/CLOCKS_PER_SEC);
}}

void freeHashTable() {
int i;
HashNode *current,*next;
for(i=0;i<TABLE_SIZE;i++) {
current=hashTable[i];
while(current!=NULL) {
next=current->next;
free(current);

```

```

current=next;
}}}
int main() {
Event events[]={
{"172.16.0.8",100,"Malware Alert","10:00:20",1},
{"192.168.1.10",95,"Brute Force Login","10:00:10",2},
{"192.168.1.10",90,"Login Failed","10:00:01",3},
{"192.168.1.20",85,"Port Scan","10:00:05",4},
{"192.168.1.20",80,"Port Scan","10:00:05",5},
{"10.0.0.5",20,"Normal Login","10:00:15",6}};
int n=sizeof(events)/sizeof(events[0]);
int i;
AVLNode *root=NULL;
MaxHeap heap={.size=0};
initHashTable();
for(i=0;i<n;i++) {
insertIP(events[i].ip);
root=insertAVL(root,events[i]);
insertHeap(&heap,events[i]);}
printDuplicateReport();
printf("\n--- AVL RISK ORDER ---\n");
printAVL(root);
Event *top=highestRisk(&heap);
if(top!=NULL)
printf("\nHighest Risk: %s | Score: %d\n",
top->ip,top->risk);
performanceTest();
printf("\nHash Collisions: %d\n",hashCollisions);
freeHashTable();
return 0;
}

```

Output Screen Shot

```
=====
REAL-TIME CYBER THREAT DETECTION ENGINE
=====

Total Events Processed : 6

----- DUPLICATE IP DETECTION REPORT -----
192.168.1.20    -> 2 occurrences
192.168.1.10    -> 2 occurrences

---- AVL TREE : EVENTS ORDERED BY RISK (Highest Risk First) ----
IP Address      | Risk | Event Type      | Timestamp
-----
172.16.0.8      | 100  | Malware Alert   | 10:00:20
192.168.1.10    | 95   | Brute Force Login | 10:00:10
192.168.1.10    | 90   | Login Failed    | 10:00:01
192.168.1.20    | 85   | Port Scan       | 10:00:05
192.168.1.20    | 80   | Port Scan       | 10:00:05
10.0.0.5        | 20   | Normal Login    | 10:00:15

--- PRIORITY QUEUE : HIGH RISK EVENTS (Highest Risk First) ---
Rank | IP Address      | Risk | Event Type      | Timestamp
-----
1    | 172.16.0.8      | 100  | Malware Alert   | 10:00:20
2    | 192.168.1.10    | 95   | Brute Force Login | 10:00:10
3    | 192.168.1.10    | 90   | Login Failed    | 10:00:01
...

----- HASH TABLE PERFORMANCE TEST (Insertion Time) -----
Number of Events | Insertion Time (Sec)
-----
100              | ...
500              | ...
1000             | ...
2000             | ...
5000             | ...

----- COMPLEXITY SUMMARY -----
Hash Table (Search/Insert)      : O(1) average case
AVL Tree (Insert/Search)        : O(log n)
Priority Queue (Insert)          : O(log n)
Priority Queue (DeleteMax)       : O(log n)

Hash Table Size                  : 17 buckets
Hash Collisions                  : ...

-----

Program finished successfully.
```

Fig: 2 Output Screen Shot

4. Use of Modern Tools

Tool	Purpose
C / GCC	Development and compilation of the threat detection engine
Visual Studio Code	Writing, editing, debugging and managing the C source code
GitHub	Storing and submitting source code, pseudocode and project results
Terminal	Compiling, executing and testing the program
clock()	Measuring execution time during performance benchmarking

The implementation can be compiled using GCC and executed through the terminal. Visual Studio Code is used for source-code development, while GitHub provides a platform for maintaining and submitting the project files. Screenshots of compilation, execution, source code and outputs can be included as evidence of tool usage.

5. Results and Validation

Sample Security Events

IP Address	Event Type	Risk Score
192.168.1.10	Login Failed	90
192.168.1.20	Port Scan	80
192.168.1.10	Brute Force Login	95
10.0.0.5	Normal Login	20
172.16.0.8	Malware Alert	100
192.168.1.20	Port Scan	85

Duplicate IP Detection

The system identifies repeated source IP addresses while processing the events.

IP Address	Occurrences
192.168.1.10	2
192.168.1.20	2

Risk-Based Output

The AVL Tree and Max Heap are used to organize and prioritize events according to their risk scores.

Rank	IP Address	Risk	Event
1	172.16.0.8	100	Malware Alert
2	192.168.1.10	95	Brute Force Login
3	192.168.1.10	90	Login Failed
4	192.168.1.20	85	Port Scan
5	192.168.1.20	80	Port Scan
6	10.0.0.5	20	Normal Login

Performance Test

Number of Events	Measured Time (sec)
100	Record from program
500	Record from program
1,000	Record from program
2,000	Record from program
5,000	Record from program

The actual execution-time values will be obtained from the program output. The benchmark helps evaluate how processing time changes as the number of security events increases.

6. Analysis and Engineering Decision

Approach	Advantage	Limitation
Linear Search	Easy to implement	$O(n)$ search
Hash Table	Fast average IP lookup	Collisions require additional handling
Normal BST	Maintains ordered data	May become unbalanced
AVL Tree	Maintains balanced height	Rotations increase implementation complexity
Max Heap	Efficient highest-risk retrieval	Not suitable for complete sorted traversal

The hash table with separate chaining was selected for IP lookup and duplicate detection because it provides efficient average-case access. An AVL Tree was selected to maintain balanced risk-based organization and avoid the unbalanced behaviour of a normal BST. The Max Heap was selected because the system needs quick access to the highest-risk security event. Combining these structures allows each operation to be handled using the most suitable data structure.

7. Broader Considerations

Sustainability

The use of efficient data structures reduces unnecessary searching and processing, which supports efficient handling of increasing security-event volumes.

Society and Safety

Quick identification of high-risk activities can help organizations respond to suspicious behaviour and protect digital systems, services and users.

Ethics and Privacy

Cybersecurity logs may contain sensitive information such as IP addresses and activity details. Therefore, the system should be used only for authorized monitoring, testing and security analysis.

Professional Responsibility

The results should be validated carefully because incorrect classification may produce unnecessary alerts or fail to identify important threats. Proper testing, documentation and responsible handling of security data are therefore essential.

8. Conclusion

The project successfully developed a **Real-Time Cyber Threat Detection Engine** using suitable data structures for efficient security-event processing. The proposed solution combines a **Hash Table with separate chaining, AVL Tree, and Max Heap** to perform IP lookup, duplicate detection, risk-based organization, and priority-based threat identification.

The major finding is that using different data structures for different operations provides an efficient and organized approach to handling cybersecurity events. The system can identify repeated IP addresses, maintain risk information, retrieve high-risk events, and measure performance for different event volumes.

The major functional and performance requirements were addressed through implementation and testing. However, the current system has limitations such as the use of controlled security-event data and a limited testing environment. The performance results may also vary depending on the input distribution and system resources.

With additional time and resources, the system could be improved by supporting **larger real-time datasets, advanced threat classification, more detailed visualization, automated alert generation, and additional performance optimization**. Integration with live security logs could further improve its practical application.

9. Student Reflection

This assignment provided practical understanding of how data structures can be applied to solve a real-world cybersecurity problem. Beyond classroom theory, the project helped in understanding the interaction between **hashing, collision handling, AVL Tree balancing, and priority queues** within a single application. It also provided experience in performance measurement, debugging, testing, and interpreting experimental results.

The project also improved problem-solving and design skills by requiring the selection of suitable data structures for different operations. If additional time and resources were available, the system could be extended with **real-time data sources, larger datasets, improved threat analysis, and a more advanced monitoring interface**. These improvements would make the project more suitable for practical cybersecurity monitoring while providing better scalability and accuracy.

10. References

1. Pascoe, C., Quinn, S., & Scarfone, K. (2024). *The NIST Cybersecurity Framework (CSF) 2.0*. National Institute of Standards and Technology.
2. National Institute of Standards and Technology. (2024). *Cybersecurity Framework 2.0 Resource Center*. NIST.
3. Cybersecurity and Infrastructure Security Agency. (2024). *Cross-Sector Cybersecurity Performance Goals (CPGs)*. CISA.
4. Python Software Foundation. (2026). *heapq — Heap queue algorithm*. Python 3.14 Documentation.
5. Pallets Projects. (2026). *Flask Documentation — Flask 3.1.x*.
6. GitHub. (2026). *Repositories Documentation*. GitHub Docs.
7. GitHub. (2026). *Creating and Managing Repositories*. GitHub Docs.
8. National Institute of Standards and Technology. (2024). *NIST Cybersecurity Framework (CSF) 2.0*. NIST Cybersecurity and Privacy Reference Tool.
9. Cybersecurity and Infrastructure Security Agency. (2024). *Cybersecurity Performance Goals: Frequently Asked Questions*. CISA.
10. Python Software Foundation. (2026). *Heap Queue Algorithm — Priority Queue Implementation Notes*. Python Documentation.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTA	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2	PO1, PO2	L3/L4	10
Application of knowledge	CO2, CO3	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO3, CO4	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO4, CO5	PO5	L3/L4	10
Validation	CO5	PO2, PO4	L4/L5	15
Analysis & justification	CO3, CO5	PO2, PO3	L4/L5	15
Broader considerations	CO5	PO6, PO7	L4/L5	5
Documentation & reflection	CO5	PO10	L3/L4	5

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis: Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____

